

Bài 4: NGĂN XẾP, HÀNG ĐỢI

Mục tiêu:

Về kiến thức:

+ Hiểu được kiểu dữ liệu trừu tượng Ngăn xếp: Khái niệm ngăn xếp, các dạng biểu diễn ngăn xếp, các phép toán cơ bản trên ngăn xếp, ứng dụng của ngăn xếp.

+ Hiểu được kiểu dữ liệu trừu tượng Hàng đợi: Khái niệm hàng đợi, các dạng biểu diễn hàng đợi, các phép toán cơ bản trên hàng đợi, ứng dụng của hàng đợi.

Về kỹ năng:

+ Vận dụng kiến thức đã học để cài đặt các phép toán cơ bản trên ngăn xếp và hàng đợi trong các bài toán cụ thể

4.1 Ngăn xếp

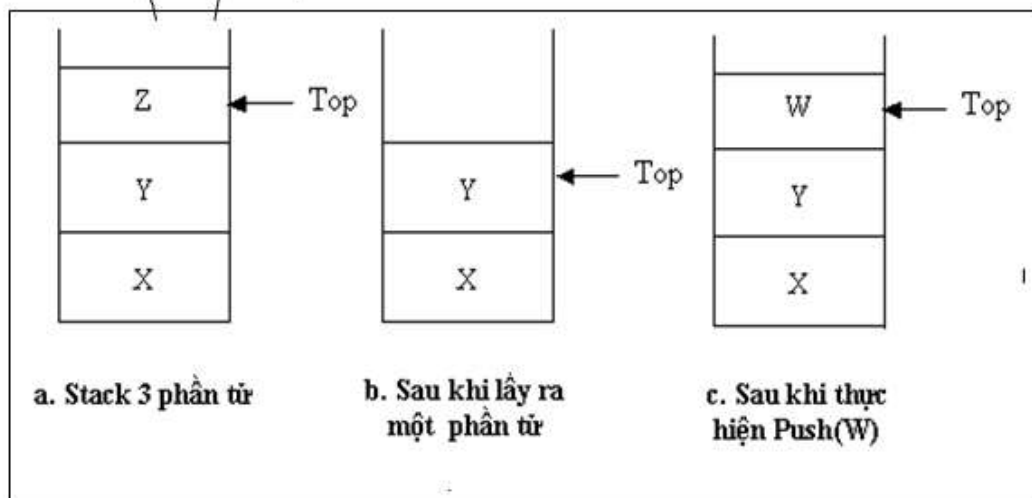
4.1.1 Định nghĩa

Ngăn xếp (Stack) là một danh sách đặc biệt, trong đó việc thêm vào hoặc loại bỏ một phần tử chỉ thực hiện tại một đầu của danh sách, đầu này gọi là đỉnh (TOP) của ngăn xếp.

Ví dụ: Ta có thể xem hình ảnh trực quan của ngăn xếp bằng một chồng đĩa đặt trên bàn. Muốn thêm vào chồng đó 1 đĩa ta để đĩa mới trên đỉnh chồng, muốn lấy các đĩa ra khỏi chồng ta cũng phải lấy đĩa trên đỉnh trước.

=> Như vậy ngăn xếp là một cấu trúc có tính chất “vào sau - ra trước” hay “vào trước – ra sau” (**LIFO**: Last In - First Out hay **FILO**: First In – Last Out)

- Hình ảnh của ngăn xếp có dạng (hình 4.1.a) việc lấy một phần tử ra khỏi ngăn xếp mô tả như hình 4.1.b, việc thêm một phần tử vào ngăn xếp được mô tả như hình 4.1.c:



Hình 4.1: Cấu trúc ngăn xếp

4.1.2 Biểu diễn ngăn xếp trên máy tính

Tương tự danh sách, ngăn xếp có thể biểu diễn bởi mảng hoặc con trỏ, ví dụ về biểu diễn ngăn xếp bởi mảng như sau:

```
#include<stdio.h>
#define N 100
typedef int item;
typedef struct Stack{
    item elems[N];
    int top; // chỉ vị trí thêm vào và lấy ra
}Stack;
```

4.1.3 Các thao tác trên ngăn xếp

1- Tạo ngăn xếp rỗng

Với phép toán khởi tạo ngăn xếp rỗng, ta nên để chỉ số $top = -1$ để khi thêm phần tử đầu tiên vào ngăn xếp, top tăng lên 1 đơn vị và chỉ vào phần tử đầu tiên.

```
// phép toán khởi tạo
void init_S(Stack *S){
    S->top=-1;
}
```

2 - Kiểm tra tính rỗng, tính đầy của ngăn xếp

Ngăn xếp rỗng khi $top = -1$ và đầy khi $top = N-1$

```
// pheps toan kiem tra ngan xep rong
int isEmpty_S(Stack S){
    if(S.top == -1)
        return 1;
    else
        return 0;
}
// Kiem tra ngan xep day
int isFull_S(Stack S){
    if(S.top == N-1)
        return 1;
    else
        return 0;
}
```

3 - Thêm phần tử vào ngăn xếp

Để thêm một phần tử vào đỉnh ngăn xếp, ta tăng chỉ số top lên 1 đơn vị, sau đó đẩy phần tử cần thêm vào top.

```
// Day 1 phan tu x vao ngan xep S
int push(item x, Stack *S){
    if (!isFull_S(*S)){
        S->top++;
        S->elems [S->top] = x;
    } else
        printf("\n NGan xep day.");
}
```

4 – Lấy 1 phần tử ra khỏi ngăn xếp

Để lấy 1 phần tử ra khỏi ngăn xếp S và lưu vào biến x, trước hết ta lưu lại giá trị tại top vào biến x, sau đó giảm top đi 1 đơn vị.

```
// Nhac 1 ptu ra khoi ngan xep
int pop(Stack *S, item *x){
    if (!isEmpty_S(*S)){
        *x = S->elems [S->top];
        S->top--;
    }
}
```

4.2 Hàng đợi

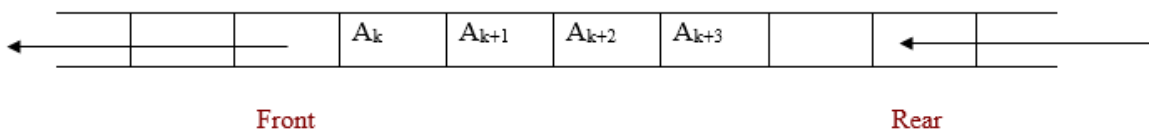
4.2.1 Định nghĩa

Hàng đợi, hay ngăn gọn là hàng (queue) là một danh sách đặc biệt mà phép thêm một

phần tử vào hàng chỉ thực hiện tại một đầu của hàng, gọi là cuối hàng (Rear), còn phép lấy một phần tử ra khỏi hàng thì thực hiện ở đầu còn lại của hàng, gọi là đầu hàng (Front).

*Ví dụ: Khi ta xếp hàng mua vé xem phim là một hình ảnh trực quan của khái niệm trên, người mới đến thêm vào cuối hàng còn người ở đầu hàng mua vé và ra khỏi hàng, vì vậy hàng còn được gọi là cấu trúc hoạt động theo nguyên tắc **FIFO**: First In - First Out, hay theo nguyên tắc "vào trước - ra trước, vào sau – ra sau".*

Hình ảnh của hàng có dạng như sau:



Hình 4.2: Hình ảnh hàng đợi

4.2.2 Biểu diễn hàng đợi trên máy tính

Như đã trình bày trong phần ngăn xếp, ta hoàn toàn có thể dùng danh sách để biểu diễn cho một hàng và dùng các phép toán đã được cài đặt của danh sách để cài đặt các phép toán trên hàng. Tuy nhiên làm như vậy có khi sẽ không hiệu quả, chẳng hạn dùng danh sách cài đặt bằng mảng ta thấy lời gọi `INSERT_LIST(x, ENDLIST(Q), Q)` tốn một hằng thời gian trong khi lời gọi `DELETE_LIST(FIRST(Q), Q)` để xóa phần tử đầu hàng (phần tử ở vị trí 0 của mảng) ta phải tốn thời gian tỉ lệ với số các phần tử trong hàng để thực hiện việc dời toàn bộ hàng lên một vị trí. Để cài đặt hiệu quả hơn ta phải có một suy nghĩ khác dựa trên tính chất đặc biệt của phép thêm và loại bỏ một phần tử trong hàng, Đó là sử dụng trực tiếp mảng và con trỏ để cài đặt hàng đợi được trình bày sau đây.

4.2.2.1 Cài đặt hàng đợi sử dụng mảng

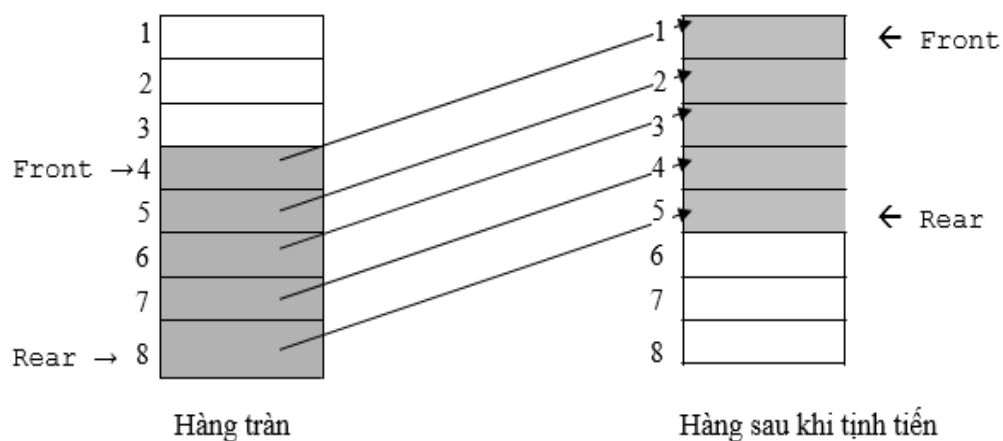
Ta dùng một mảng để chứa các phần tử của hàng (giả sử hàng có tối đa n phần tử, kiểu dữ liệu của các phần tử trong hàng là Item), khởi đầu, phần tử đầu tiên của hàng được đưa vào vị trí đầu tiên của mảng, phần tử thứ 2 vào vị trí tiếp theo của mảng... Giả sử hàng có n phần tử, ta có $\text{front} = 0$ và $\text{rear} = n-1$.

- + Khi xoá một phần tử front tăng lên 1,
- + Khi thêm một phần tử rear tăng lên 1.

Như vậy hàng có khuynh hướng đi xuống, đến một lúc nào đó ta không thể thêm vào hàng được nữa ($\text{rear} = n-1$) dù mảng còn nhiều chỗ trống (các vị trí trước front) trường hợp này ta gọi là *hàng bị tràn*. Trong trường hợp toàn bộ mảng đã chứa các phần tử của hàng ta gọi là *hàng bị đầy*.

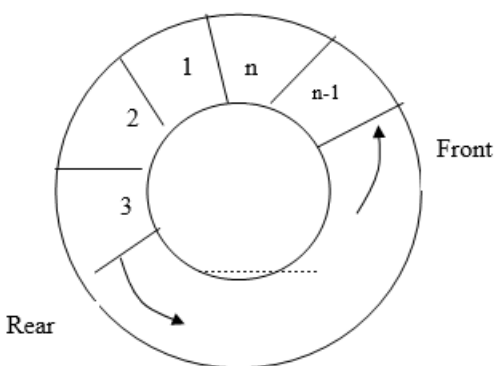
Các cách khắc phục hàng bị tràn

1 - Dời toàn bộ hàng lên front -1 vị trí: cách này gọi là di chuyển tịnh tiến. Trong trường hợp này ta luôn có $\text{front} \leq \text{rear}$. (**Hình 4.3**, minh họa cho cách này)



Hình 4.3: Hàng dời tịnh tiến

2 - Xem mảng như là một vòng tròn: Nghĩa là khi hàng bị tràn nhưng chưa đầy ta thêm phần tử mới vào vị trí 1 của mảng, thêm một phần tử mới nữa thì thêm vào vị trí 2 (nếu có thể)... Rõ ràng cách làm này front có thể lớn hơn rear. Cách khắc phục này gọi là dùng mảng xoay vòng (xem **Hình 4.4**). Tổ chức hàng tròn tức là ta cho phần tử thứ 1 đứng sau phần tử thứ n trong hàng.



Hình 4.4: Hàng đợi vòng tròn

Dạng biểu diễn hàng đợi bằng mảng tròn

```
#include<stdio.h>
#define N 100
typedef int item;
//hàng đợi ghép nối vòng
typedef struct Queue{
    item elem[N]; // mảng lưu phần tử trong hàng đợi
    int size; // lưu số lượng ptu trong hàng đợi
    int front, rear; // chỉ số lấy ra, thêm vào trong hàng đợi
}Queue;
```

Các phép toán trên hàng đợi tròn

Khởi tạo hàng rỗng

Hàng đợi rỗng khi front và rear không trở đến vị trí hợp lệ nào trong mảng vậy ta có thể cho front và rear đều bằng 0, size=0. Tuy nhiên ta nên khởi tạo giá trị của Front = -1 để khi thêm phần tử đầu tiên vào hàng khi hàng đang rỗng thì Q.Front = Q.Rear = 0: Hàng có một phần tử thì Front, Rear đều trở tới phần tử này

```
void init_Q(Queue *Q){
    Q->size = 0;
    Q->front = 0;
    Q->rear = -1;
}
```

Kiểm tra tính rỗng, tính đầy của hàng đợi

```

// kiem tra hang doi rong
int isEmpty_Q(Queue Q){
    if (Q.size==0)
        return 1;
    else
        return 0;
}

// kiem tra hang doi day
int isFull_Q(Queue Q){
    if (Q.size ==N)
        return 1;
    else
        return 0;
}

```

Thêm một phần tử vào hàng đợi

Để thêm một phần tử vào cuối hàng đợi, ta thực hiện tăng chỉ số cuối hàng lên 1 đơn vị, đổ dữ liệu cần thêm vào vị trí cuối hàng rồi tăng size lên 1 đơn vị.

```

//Dua 1 phan tu vao hang doi
void enqueue(item x, Queue *Q){
    if(!isFull_Q(*Q)){
        Q->size ++;
        Q->last =(Q->last +1)%N; // %N hang doi quay vong
        Q->elem [Q->last ]=x;
    }
}

```

Xóa một phần tử khỏi hàng đợi

Để lấy 1 phần tử ra khỏi hàng đợi và lưu vào biến x, ta lưu lại giá trị ở đầu hàng vào biến x, sau đó tăng chỉ số đầu hàng lên 1 đơn vị, giảm kích thước size xuống 1 đơn vị

```

void dequeue(Queue*Q, item *x){
    if(!isEmpty_Q(*Q)){
        *x=Q->elem[Q->first];
        Q->size--;
        Q->first=(Q->first+1)%N;
    }
}

```

4.2.2.2 Cài đặt hàng đợi sử dụng con trỏ

Cách tự nhiên nhất là dùng hai con trỏ front và rear để trỏ tới phần tử đầu và cuối

hàng. Hàng được cài đặt như một danh sách liên kết đơn (kép, hoặc vòng).

Giả sử ta xét dạng cài đặt hàng bằng danh sách liên kết đơn, khi đó dạng cài đặt của hàng đợi tương tự danh sách liên kết đơn, tuy nhiên để quản lý danh sách liên kết đơn ta sử dụng một con trỏ, còn để quản lý hàng đợi ta dùng 2 con trỏ, một con trỏ quản lý đầu hàng đợi và một con trỏ quản lý cuối hàng đợi

Dạng biểu diễn hàng đợi bằng con trỏ

```
typedef int item; //kieu du lieu
struct node{
    item data;
    node * next;
};
struct Queue{
    node * front, *rear; //node dau va node cuoi
    int count; //dem so phan tu
};
```

Các phép toán trên hàng đợi biểu diễn bằng con trỏ

Tạo Node P chứa dữ liệu x

```
node *makeNode(item x) {
    node *P = (node*) malloc(sizeof(node));
    P->next = NULL;
    P->data = x;
    return P;
}
```

Thêm phần tử vào cuối hàng đợi

```
void push(Queue &Q, item x) {
    node *P = makeNode(x); //Neu Q rong
    if (isEmpty(Q)){
        Q.front = Q.rear = P; //dau va cuoi deu tro den P
    }else { //Khong rong
        Q.rear->next = P;
        Q.rear = P;
    }
    Q.count ++ ; //tang so phan tu len
}
```

Xóa phần tử ở đầu hàng đợi

```

int pop(Queue &Q){
    if (isEmpty(Q)) {
        printf("Hàng đợi rỗng !");
        return 0;
    } else{
        item x = Q.front->data;
        if (Q.count == 1) //neu co 1 phan tu
            init(Q);
        else
            Q.front = Q.front->next;
        Q.count --;
        return x; //tra ve phan tu lay ra
    }
}

```

4.3 Thực hành

4.3.1 Mục đích của bài thực hành

Bài thực hành giúp sinh viên biết cách cài đặt 2 kiểu dữ liệu trừu tượng của mô hình dữ liệu danh sách: Ngăn xếp và Hàng đợi. Sau đó, sinh viên thực hành thành thạo trên các phép toán trên Ngăn xếp và Hàng đợi. Cuối cùng, sinh viên có khả năng vận dụng vào triển khai bài toán ứng dụng.

4.3.2 Bài thực hành mẫu

Bài 1: Cho một ngăn xếp chứa các số nguyên. Yêu cầu:

1. Biểu diễn ngăn xếp bởi con trỏ
2. Thực hiện các thao tác trên ngăn xếp:
 - a. Khởi tạo ngăn xếp rỗng
 - b. Thêm một phần tử vào ngăn xếp
 - c. Lấy một phần tử ra khỏi ngăn xếp
 - d. Nhập dữ liệu cho ngăn xếp
 - e. Hiển thị dữ liệu chứa trong ngăn xếp

```

#include <stdlib.h>
#include <stdio.h>
typedef int item; //kiểu du lieu
struct Node
{
    item Data; //du lieu
    Node *Next; //link
};

```

```

typedef struct Stack
{
    Node *Top;
};
void Init (Stack &S); //khởi tạo Stack rỗng
int Isempy(Stack S); //kiểm tra Stack rỗng
int Len (Stack S); //Độ dài Stack
void Push(Stack &S, item x); //thêm phần tử vào Stack
int Peak(Stack S); //Lấy phần tử ở đầu Stack nhưng không xóa
int Pop(Stack &S); //Loại bỏ phần tử khỏi Stack
void Input (Stack &S); //Nhập Stack
void Output(Stack S); //Xuất Stack
Node *MakeNode(item x); //Tạo 1 Node
void Init (Stack &S) //khởi tạo Stack rỗng
{
    S.Top = NULL;
}
int Isempy(Stack S) //kiểm tra Stack rỗng
{
    return (S.Top == NULL);
}
int Len (Stack S)
{
    Node *P = S.Top;
    int i=0;
    while (P != NULL) //trong khi chưa hết Stack thì vẫn duyệt
    {
        i++;
        P = P->Next;
    }
    return i;
}
Node *MakeNode(item x) //tạo 1 Node
{
    Node *P = (Node*) malloc(sizeof(Node));
    P->Next = NULL;
    P->Data = x;
    return P;
}
void Push(Stack &S, item x) //thêm phần tử vào Stack
{
    Node *P = MakeNode(x);
    P->Next = S.Top;
}

```

```

    S.Top = P;
}
int Peak(Stack S) //Lay phan tu o dau Stack nhưng khong xoa
{
    return S.Top->Data;
}
int Pop(Stack &S) //Loai bo phan tu khoi Stack
{
    if (!Iseempty(S))
    {
        item x = S.Top->Data; //luu lai gia tri
        S.Top = S.Top->Next; //Xoa phan tu Top
        return x;
    }
}

void Input (Stack &S) //nhap danh sach
{
    int i=0;    item x;
    do
    {
        i++;
        printf ("Nhap phan tu thu %d : ",i);
        scanf("%d",&x);
        if (x != 0) Push(S,x);
    } while(x != 0); //nhap 0 de ket thuc
}

void Output(Stack S)
{
    Node *P = S.Top;
    while (P != NULL)
    {
        printf("%d  ",P->Data);
        P = P->Next;
    }
    printf("\n");
}

int main()
{
    Stack S;
    Init(S);
    Input(S);
    Output(S);
}

```

```

        item x;
    printf ("Nhap phan tu can chen vao DS: ");
    scanf ("%d",&x);
    Push(S,x);
    Pop(S);
}

```

4.3.3 Bài tập luyện tập

Bài 2: Cho một ngăn xếp chứa các số nguyên. Yêu cầu:

1. Biểu diễn ngăn xếp bởi mảng
2. Thực hiện các thao tác trên ngăn xếp:
 - a. Khởi tạo ngăn xếp rỗng
 - b. Thêm một phần tử vào ngăn xếp
 - c. Lấy một phần tử ra khỏi ngăn xếp
 - d. Nhập dữ liệu cho ngăn xếp
 - e. Hiển thị dữ liệu chứa trong ngăn xếp

Bài 3: Cho một Hàng đợi chứa các số nguyên. Yêu cầu:

1. Biểu diễn hàng đợi bởi con trỏ
2. Thực hiện các thao tác trên Hàng đợi:
 - a. Khởi tạo hàng đợi rỗng
 - b. Thêm một phần tử vào cuối hàng đợi
 - c. Lấy một phần tử ra khỏi đầu hàng đợi
 - d. Nhập dữ liệu cho hàng đợi
 - e. Hiển thị dữ liệu chứa trong hàng đợi

Bài 4: Cho một Hàng đợi chứa các số nguyên. Yêu cầu:

1. Biểu diễn hàng đợi bởi mảng
2. Thực hiện các thao tác trên Hàng đợi:
 - a. Khởi tạo hàng đợi rỗng
 - b. Thêm một phần tử vào cuối hàng đợi
 - c. Lấy một phần tử ra khỏi đầu hàng đợi
 - d. Nhập dữ liệu cho hàng đợi
 - e. Hiển thị dữ liệu chứa trong hàng đợi

Bài 5: Cho ngăn xếp và các phép toán cơ bản trên ngăn xếp. Viết chương trình đổi 1 số nguyên từ hệ 10 (hệ thập phân) sang các hệ 2, hệ 8, hệ 16

Bài 6: Cho một ngăn kéo/ngăn xếp chứa các cuốn sách. Yêu cầu:

1. Biểu diễn ngăn kéo đó bởi ngăn xếp cài đặt bởi con trỏ
2. Thực hiện cài đặt các thao tác sau
 - a. Thêm một cuốn sách vào ngăn kéo
 - b. Lấy một cuốn sách ở vị trí thứ k kể từ đáy ngăn kéo ra khỏi ngăn kéo sao cho trật tự các phần tử khác không đổi.
 - c. Đảo ngược thứ tự các cuốn sách trong ngăn kéo
 - d. Hoán đổi vị trí của 2 cuốn sách tại các vị trí p_1, p_2 trong ngăn kéo
 - e. Thay cuốn sách ở vị trí k trong ngăn kéo kể từ đỉnh bởi cuốn sách

Bài 7: Cho hàng đợi và các phép toán cơ bản trên hàng đợi. Yêu cầu:

1. Viết chương trình thêm 1 phần tử vào đầu hàng đợi sao cho trật tự các phần tử còn lại không thay đổi
2. Viết chương trình xóa phần tử ở cuối hàng đợi sao cho trật tự các phần tử còn lại không thay đổi